

Optimizer Matters

DS 4440 · LECTURE 7

1. Where we are

Backprop yields **gradients** for all parameters efficiently, $\nabla_W \mathcal{L}$. Why did we want those again? For **gradient descent**:

$$\hat{W}_{\tau+1} \leftarrow \hat{W}_{\tau} - \underbrace{\alpha}_{\text{learning rate}} \nabla_W \mathcal{L}(\hat{W}_{\tau}, x, y)$$

Variants

- **Stochastic**: one instance (x, y) per step.
- **Batch**: sets of (x, y) , stepping on $\frac{1}{|B|} \sum_{i \in B} \nabla_W \mathcal{L}_i$.

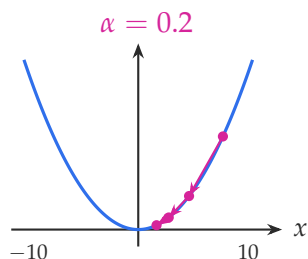
Averaging over a batch cuts the variance of the gradient estimate by roughly $1/|B|$, so the returns diminish: going from 32 to 128 halves the noise, not quarters it. The noise is not purely a nuisance — it is part of how SGD escapes flat regions.

2. The learning rate

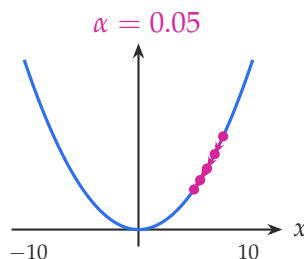
Take the smallest possible example, $f(x) = x^2$ with $\frac{d}{dx} f(x) = 2x$ and $x_{\text{init}} = 10$. The update is

$$x \leftarrow x - \alpha (2x) = (1 - 2\alpha) x,$$

so each step multiplies x by the same factor $(1 - 2\alpha)$, and the entire behaviour of gradient descent on this problem is controlled by that one number.



$$\begin{aligned} 0.2 \cdot (2 \cdot 10) &= 4 \Rightarrow x = 6 \\ 0.2 \cdot (2 \cdot 6) &= 2.4 \Rightarrow x = 3.6 \\ 0.2 \cdot (2 \cdot 3.6) &= 1.44 \Rightarrow x = 2.16 \\ \dots \end{aligned}$$



$$\begin{aligned} 0.05 \cdot (2 \cdot 10) &= 1 \Rightarrow x = 9 \\ 0.05 \cdot (2 \cdot 9) &= 0.9 \Rightarrow x = 8.1 \\ 0.05 \cdot (2 \cdot 8.1) &= 0.81 \Rightarrow x = 7.29 \\ \dots &\text{ sloooooow} \end{aligned}$$

Iterating is then just repeated multiplication. Write $r = 1 - 2\alpha$; after τ steps

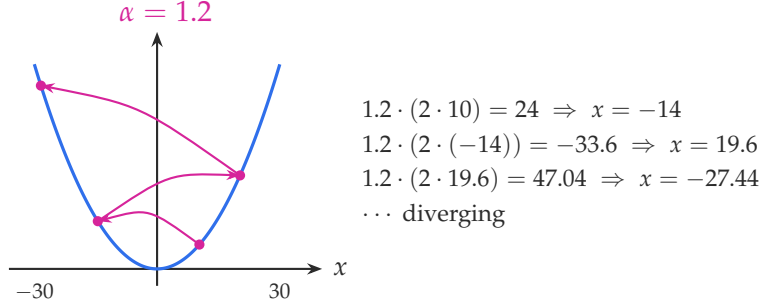
$$x_{\tau} = r^{\tau} x_0,$$

so the iterates go to zero exactly when r^{τ} does, which needs $|r| < 1$. Solving $|1 - 2\alpha| < 1$ gives $0 < \alpha < 1$.

The magnitude of r and its sign do different jobs. $|r|$ sets how fast we approach the minimum; the sign sets which side we land on. Three cases:

- $\alpha = 0.8, r = -0.6$: $10, -6, 3.6, -2.16, \dots$ — flips sides each step, still shrinking.
- $\alpha = 1, r = -1$: $10, -10, 10, \dots$ — cycles forever, never gets anywhere.
- $\alpha = 1.2, r = -1.4$: $10, -14, 19.6, \dots$ — grows without bound.

Drawn on the same axes as before, with a wider range so the iterates fit:



So too large an α does not just slow things down, it diverges.

None of this was special to $f(x) = x^2$. For a general quadratic $f(x) = \frac{1}{2}ax^2$ the derivative is ax , so the update is

$$x \leftarrow x - \alpha(ax) = \underbrace{(1 - \alpha a)}_r x,$$

and the same requirement $|r| < 1$ gives

$$-1 < 1 - \alpha a < 1 \quad \implies \quad 0 < \alpha < \frac{2}{a}.$$

Our example is the case $a = 2$, which is why the factor was $1 - 2\alpha$ and the bound was $\alpha < 1$. The bound is set by the *curvature* a , not by how far we start from the minimum. Note also that the best rate is not the largest stable one: $\alpha = 1/a$ makes $r = 0$ and lands on the minimum in a single step, while both smaller and larger values take many.

3. Why plain gradient descent zig-zags

Real problems do not have one curvature, they have one per direction. Take

$$f(w_1, w_2) = \frac{1}{2}(aw_1^2 + bw_2^2), \quad a \gg b.$$

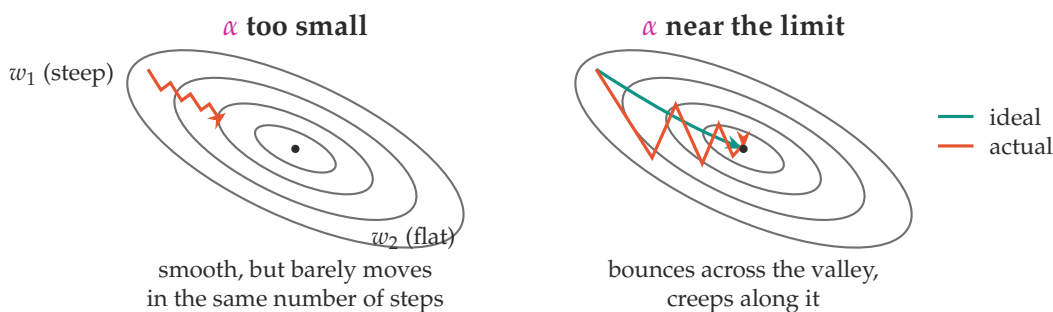
The coordinates do not interact, so gradient descent runs the previous analysis twice in parallel — one factor r per direction, but only one α to set them both:

$$w_1 \leftarrow \underbrace{(1 - \alpha a)}_{r_1} w_1, \quad w_2 \leftarrow \underbrace{(1 - \alpha b)}_{r_2} w_2.$$

Now the two requirements fight. Both need $|r| < 1$, but $a > b$, so it is r_1 that binds: stability in the steep direction needs $\alpha < 2/a$. But with $\alpha \approx 1/a$, the flat direction only shrinks by a factor of

$$1 - \alpha b \approx 1 - \frac{b}{a} = 1 - \frac{1}{\kappa}, \quad \kappa \stackrel{\text{def}}{=} \frac{a}{b} \quad (\text{the condition number})$$

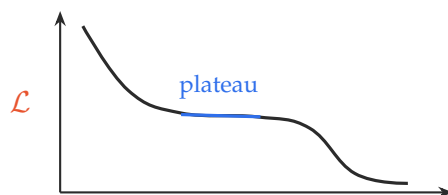
per step. With $\kappa = 100$ we need on the order of a hundred steps to make progress along the flat direction, while the steep direction is already done. Turning α up to fix that pushes r_1 below -1 and the steep direction blows up. The steps bounce across the narrow valley and creep along it.



Neither setting works, and nothing in between fixes it: too small and only the steep direction makes progress, too large and it diverges, and in the usable range the path zig-zags. One α cannot serve two curvatures. This is what makes basic gradient descent *myopic* — it follows the steepest local direction, which is not the direction of the minimum unless the contours are round. **Normalizing inputs** helps because it makes the curvatures more similar — it shrinks κ . Everything in the rest of this lecture is a way of coping with $\kappa \gg 1$ without computing second derivatives.

A general challenge: saddle points

A critical point ($\nabla = 0$) is a minimum only if the curvature is positive in *every* direction. In d dimensions that is d conditions, so in a large network critical points are overwhelmingly saddles rather than minima. The two-dimensional example is $f(x, y) = x^2 - y^2$ at the origin: a minimum along x , a maximum along y .



In practice the trouble is less the saddle itself than the flat region around it, where gradients are tiny and progress stalls for many steps.

4. Momentum

Keep going in the direction you have been going:

$$v_\tau \leftarrow \gamma v_{\tau-1} - \alpha \nabla_W \mathcal{L}(\hat{W}_\tau, x, y), \quad \hat{W}_{\tau+1} \leftarrow \hat{W}_\tau + v_\tau$$

with $\gamma = 0.9$ typical. In the zig-zag picture, the components that keep reversing sign cancel out across steps while the consistent component along the valley accumulates.

It is worth knowing how much it accumulates. If the gradient were constant at g , then v settles at

$$v_\infty = \gamma v_\infty - \alpha g \quad \implies \quad v_\infty = -\frac{\alpha g}{1 - \gamma},$$

so momentum multiplies the effective step size by $1/(1 - \gamma)$ — a factor of 10 at $\gamma = 0.9$. That is why adding momentum usually means lowering α .

Nesterov momentum

We can anticipate where we are going: evaluate the gradient at the point the momentum term is about to take us to, rather than where we are.

$$v_\tau \leftarrow \gamma v_{\tau-1} - \alpha \nabla_W \mathcal{L}(\underbrace{\hat{W}_\tau + \gamma v_{\tau-1}}_{\text{move toward the accumulated step first}}, x, y)$$

If the step is about to overshoot, the gradient at the lookahead point already points back, and the correction happens one step earlier. (See examples in Colab.)

5. One learning rate per parameter

So far a single α for all parameters. Write $g_{\tau,j}$ for the j -th component of the gradient at step τ , and allow a per-parameter rate:

$$g_{\tau,j} \stackrel{\text{def}}{=} [\nabla_W \mathcal{L}(\hat{W}_\tau)]_j, \quad \hat{W}_{(\tau+1),j} \leftarrow \hat{W}_{\tau,j} - \alpha_{\tau,j} g_{\tau,j}.$$

AdaGrad

Move slower for parameters that have already been updated a lot:

$$\alpha_{\tau,j} \stackrel{\text{def}}{=} \frac{\alpha}{\sqrt{G_{\tau,j}} + \epsilon}, \quad G_{\tau,j} = \sum_{t \leq \tau} g_{t,j}^2,$$

with ϵ there to avoid dividing by zero. Rare features get large steps, frequent ones get small steps. The catch: $G_{\tau,j}$ is a sum of squares, so it only ever grows, and $\alpha_{\tau,j} \rightarrow 0$. Given enough steps, AdaGrad stops learning whether or not it has arrived.

RMSProp

Fix that by replacing the running sum with a **decaying average**, so old gradients fade out:

$$S_{\tau,j} \stackrel{\text{def}}{=} \lambda S_{(\tau-1),j} + (1 - \lambda) g_{\tau,j}^2$$

and then divide α by $\sqrt{S_{\tau,j}}$. The denominator can now go down as well as up, so a parameter that was busy early is not penalized forever.

Adam

Probably the go-to optimizer now. Keep two running averages per parameter — momentum on the gradient, RMSProp on its square:

$$\begin{aligned} m_\tau &\leftarrow \beta_1 m_{\tau-1} + (1 - \beta_1) g_\tau && \approx \text{momentum} \\ v_\tau &\leftarrow \beta_2 v_{\tau-1} + (1 - \beta_2) g_\tau^2 && \approx \text{RMSProp} \end{aligned}$$

then

$$\hat{W}_{(\tau+1),j} \leftarrow \hat{W}_{\tau,j} - \frac{\alpha}{\sqrt{v_{\tau,j}} + \epsilon} m_{\tau,j}.$$

Bias correction, which that version omits. Both averages start at zero, so with $\beta_1 = 0.9$ the first step gives $m_1 = (1 - \beta_1)g_1 = 0.1 g_1$ — ten times too small. Dividing by $(1 - \beta_1^\tau)$ undoes exactly this, and since $\beta_1^\tau \rightarrow 0$ the correction fades as training proceeds:

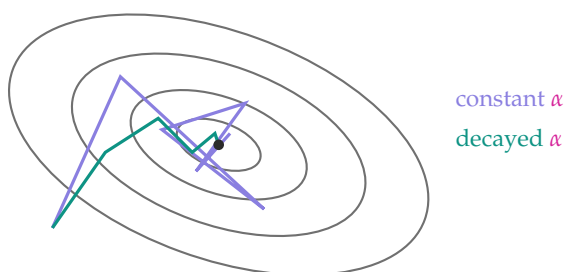
$$\hat{m}_\tau = \frac{m_\tau}{1 - \beta_1^\tau}, \quad \hat{v}_\tau = \frac{v_\tau}{1 - \beta_2^\tau}.$$

optimizer	state per parameter	what it fixes
SGD	none	—
+ momentum	v	zig-zag, small consistent gradients
AdaGrad	G	one rate does not fit all parameters
RMSProp	S	AdaGrad's rate decaying to zero
Adam	m, v	both of the above

Adam keeps two extra numbers per parameter, so optimizer state costs about twice the model itself in memory. Typical defaults: $\alpha = 3 \times 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. SGD with momentum still often generalizes better on vision tasks — Adam is the default, not the answer.

6. Learning-rate decay

Big steps to start, smaller steps over time. With noisy minibatches a large α that was useful early will bounce around the minimum rather than settle into it.



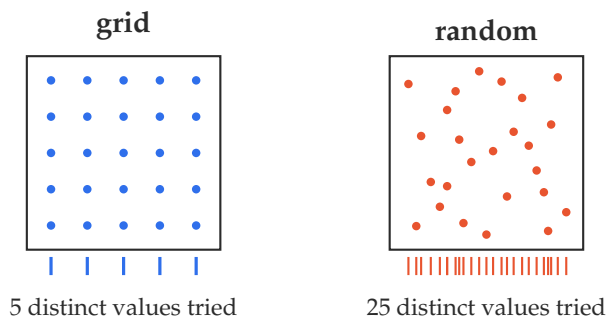
Two common schedules:

$$\alpha = \frac{1}{1 + d \cdot \text{epoch}} \alpha_0 \quad \text{and} \quad \alpha = (0.9)^{\text{epoch}} \alpha_0,$$

with d the decay rate. You will also see cosine decay, and **warmup** — starting small and ramping up over the first few hundred steps — which exists because Adam's v_τ is unreliable before it has seen many gradients.

7. A note on hyperparameter tuning

Choose **random** values, not a grid.



Both panels spend the same budget of 25 runs. Suppose only one of the two hyperparameters matters, and you do not know which. The grid tries just 5 distinct values of it, since each is repeated five times with different settings of the irrelevant one. Random search tries 25.

One practical note: sample learning rates *log-uniformly*. Drawing uniformly from $[10^{-5}, 10^{-1}]$ puts 90% of the samples above 10^{-2} ; drawing the exponent uniformly from $[-5, -1]$ covers each order of magnitude equally.